U05 · POO avanzada

Ya sabes definir una clase. Ahora toca la parte que de verdad distingue a un programador que "usa" la orientación a objetos de uno que **piensa** en orientación a objetos: relacionar clases entre sí.

1. Relaciones entre clases: ES-UN vs. TIENE-UN

Antes de escribir código, pregúntate qué tipo de relación hay entre dos clases:

- **Herencia ("ES-UN")**: `Coche extends Vehiculo` — un coche **es un** tipo más específico de vehículo.
- **Composición/agregación ("TIENE-UN")**: `Coche` **tiene un** `Motor` (como atributo) — un coche no es un tipo de motor.

⚠ El error clásico: herencia donde tocaba composición

```
// MAL — un Coche no ES UN Motor
public class Coche extends Motor {
    public void arrancar() {
        encender();  // hereda encender() de Motor, pero conceptualmente no
tiene sentido
    }
}

// BIEN — un Coche TIENE UN Motor
public class Coche {
    private Motor motor = new Motor();
    public void arrancar() {
        motor.encender();
    }
}
```

La pregunta que resuelve la duda siempre es la misma: *¿la clase hija "es un tipo de" clase padre?* Si la respuesta es no (aunque te ahorres algo de código reutilizando métodos), la relación correcta es composición, no herencia.

2. Herencia

Java implementa la herencia con `extends`. Una subclase hereda todos los miembros **no estáticos** de su superclase (salvo los constructores):

```java
class Animal {
    public void comer() {
        System.out.println("El animal come");
    }
}

class Perro extends Animal {
    public void ladrar() {
        System.out.println("Guau");
    }
}

Perro p = new Perro();
p.ladrar();   // método propio de Perro
p.comer();    // heredado de Animal
Animal a = p; // correcto: un Perro también es un Animal (polimorfismo)
```

Si no indicas `extends`, tu clase hereda implícitamente de `Object` — todas las clases de Java, tarde o temprano, descienden de ella, formando una **jerarquía de clases**. Cuanto más arriba en la jerarquía, más generales (abstractas); cuanto más abajo, más específicas.

Constructores y herencia

Los constructores **no se heredan**, pero cada constructor de una subclase empieza con una llamada implícita a `super()` — el constructor sin parámetros del padre — aunque no la escribas.

```java
class A { public A() { System.out.println("Constructor A"); } }
class B extends A { public B() { System.out.println("Constructor B"); } }
// B() en realidad es:
// public B() { super(); System.out.println("Constructor B"); }
```

⚠ El error típico de examen

```
class Persona {
    protected String nombre;
    public Persona(String nombre) { this.nombre = nombre; }   // NO hay
constructor sin parámetros
}

class Empleado extends Persona {
    protected double sueldo;
    public Empleado(double sueldo) {
        this.sueldo = sueldo;   // ERROR DE COMPILACIÓN
    }
}
```

Este código no compila: como `Persona` **no tiene** un constructor sin parámetros, la llamada implícita `super()` que Java intenta insertar en `Empleado(double)` no encuentra a quién llamar. Hay que invocar explícitamente uno de los constructores que sí existen:

```
public Empleado(String nombre, double sueldo) {
    super(nombre);       // construye primero la parte "Persona"
    this.sueldo = sueldo; // luego la parte propia de "Empleado"
}
```

Regla de oro: si la superclase no tiene constructor sin parámetros, la subclase *debe* llamar explícitamente a `super(...)` con los datos que ese constructor necesite. Y `super(...)` y `this(...)` nunca pueden coexistir en el mismo constructor — como mucho, uno de los dos, y siempre en la primera línea.

`final` en métodos y clases

- Un **método `final`** no se puede sobrescribir en ninguna subclase.
- Una **clase `final`** no se puede extender en absoluto (`String` es un ejemplo real: no puedes crear subclases de `String`).

3. Sobrescritura y polimorfismo

Cuando una subclase redefine, con la misma firma, un método que ya tenía la superclase, decimos que lo **sobrescribe** (*override*) — usa siempre la anotación `@Override` para que el compilador te avise si te equivocas en la firma:

```
class Animal {
    public String sonido() { return "..."; }
}
class Perro extends Animal {
    @Override
    public String sonido() { return "Guau"; }
}
```

El **polimorfismo** es la consecuencia más potente de esto: una variable declarada del tipo de la superclase puede referenciar objetos de cualquier subclase, y **el método que realmente se ejecuta se decide en tiempo de ejecución**, según el tipo real del objeto (no el tipo de la variable) — se llama **ligadura dinámica**.

```
Animal[] animales = { new Perro(), new Gato() };
for (Animal a : animales) {
    System.out.println(a.sonido());   // ejecuta la versión de CADA subclase real
}
```

4. Clases abstractas

Algunas clases son tan generales que nunca tiene sentido instanciarlas directamente — solo sirven como base para que otras las extiendan. Se declaran con `abstract`:

```java
public abstract class Figura {
    protected String color;

    public abstract double calcularArea();   // sin cuerpo: cada subclase lo
implementa a su manera

    public void mostrarColor() {              // sí implementado: se hereda tal cual
        System.out.println("Color: " + color);
    }
}

public class Circulo extends Figura {
    private double radio;
    @Override
    public double calcularArea() { return Math.PI * radio * radio; }
}
```

- No se puede hacer `new Figura()` — el compilador lo rechaza.
- Si una clase tiene **al menos un** método abstracto, la clase entera debe declararse `abstract`.
- Toda subclase debe implementar todos los métodos abstractos heredados, o ser abstracta ella misma también.

Es el patrón perfecto cuando sabes que **todas** las subclases necesitan un método (`calcularArea()`, en este ejemplo), pero cada una lo calcula de forma distinta y no hay una implementación común razonable que ofrecer.

5. Interfaces

Una interface lleva la idea de “clase abstracta” al extremo: **solo** define comportamiento (métodos sin implementar, más algunas constantes), y no está ligada a la herencia — una clase puede implementar varias interfaces aunque solo pueda extender una única superclase.

```java
public interface Depredador {
    void cazar();
}
public interface Presa {
    void huir();
}

public class Rana implements Depredador, Presa {   // caza insectos Y puede ser cazada
    public void cazar() { System.out.println("La rana caza un insecto"); }
    public void huir() { System.out.println("La rana escapa de una garza"); }
}
```

La diferencia conceptual con la herencia: `Rana extends Animal` dice **qué es** una rana; `implements Depredador, Presa` dice **qué es capaz de hacer**. Por eso muchas interfaces de Java terminan en "-able" (`Comparable`, `Runnable`, `Serializable`): expresan una capacidad, no una identidad.

Todos los miembros de una interface son `public` de forma implícita; sus atributos son siempre `public static final` (constantes), y sus métodos son `abstract` salvo que indiques lo contrario.

> ⓘ **¿Clase abstracta o interface?**
>
> Usa una **clase abstracta** cuando las subclases comparten código real (atributos, métodos ya implementados) además del "contrato". Usa una **interface** cuando solo quieres exigir un comportamiento a clases que pueden no tener ningún otro parentesco entre sí (una `Rana` y un `SistemaDeAlarma` no tienen nada en común, pero ambos podrían implementar `Activable`).

6. Polimorfismo y conversión de objetos

Puedes asignar un objeto de una subclase a una variable de tipo superclase sin ningún casting (**upcasting**, siempre seguro):

```java
Animal a = new Perro();   // upcasting implícito
```

Pero para volver a tratarlo como el tipo específico (**downcasting**), necesitas un casting explícito — y es buena práctica comprobar antes con `instanceof`:

```
if (a instanceof Perro) {
    Perro p = (Perro) a;   // downcasting, ahora seguro porque ya comprobamos el tipo
    p.ladrar();
}
```

7. Clases anidadas (una introducción)

Java permite definir una clase **dentro** de otra, cuando esa clase solo tiene sentido en el contexto de la que la contiene:

- **Clase anidada estática**: no necesita un objeto de la clase externa para existir.
- **Clase interna (no estática)**: vive ligada a un objeto concreto de la clase externa.
- **Clase local**: definida dentro de un método, solo visible ahí.
- **Clase anónima**: se define e instancia a la vez, sin nombre — típicamente para implementar una interface "al vuelo", en un único uso puntual. Las expresiones lambda (siguiente apartado) han sustituido a las clases anónimas en la mayoría de los casos donde la interface tiene un único método.

8. Expresiones lambda

Cuando una interface tiene **un único método abstracto** (se llama **interface funcional**, y puede marcarse con `@FunctionalInterface` para que el compilador lo verifique), Java te permite implementarla con una sintaxis mucho más breve que una clase anónima: una **expresión lambda**.

```
(parámetros) -> cuerpo
```

```
List<Integer> numeros = new ArrayList<>(List.of(1, 5, 8, 15, 50));
numeros.removeIf(i -> i % 2 == 0);   // "i" no necesita tipo explícito: Java lo
infiere
```

`i -> i % 2 == 0` es, en la práctica, una función anónima: recibe `i` y devuelve el resultado de esa expresión. Puedes pasarla como argumento, guardarla en una variable o devolverla desde un método — tratando el propio comportamiento como un dato más.

Las interfaces funcionales más usadas (`java.util.function`)

Interface	Método	Para qué sirve
`Predicate<T>`	`boolean test(T t)`	Comprobar si algo cumple una condición
`Consumer<T>`	`void accept(T t)`	Hacer algo con un valor, sin devolver nada
`Function<T,R>`	`R apply(T t)`	Transformar un valor de tipo T en uno de tipo R
`Supplier<T>`	`T get()`	Producir un valor, sin recibir ninguno

```
Predicate<String> noVacio = s -> !s.isEmpty();
Function<String, Integer> longitud = s -> s.length();
Consumer<String> imprimir = s -> System.out.println(s);
```

Referencias a métodos (`::`)

Cuando una lambda simplemente llama a un método que ya existe, puedes abreviarla aún más con `::`:

```
lista.forEach(s -> System.out.println(s));   // lambda
lista.forEach(System.out::println);           // equivalente, con referencia a método
```

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA4. Desarrolla programas utilizando objetos y clases…	g) definir y usar clases heredadas · h) métodos estáticos · i) interfaces · j) conjuntos y librerías de clases

RA	Criterios de evaluación cubiertos
RA7. Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos…	a) herencia, superclase, subclase · b) modificadores que bloquean/fuerzan la herencia · c) constructores y herencia · d) sobrescritura en clases heredadas · e) diseño de jerarquías · f) prueba y depuración de jerarquías · g) programas con jerarquías de clases · h) comentado y documentado · i) escenarios de uso de interfaces · j) herencia vs. composición

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras